Chapter 9

Stacks and Queues

How to read these notes: anything marked with the red NOT TESTED tag is included for completeness but is not required for this class. Per the course directions, this chapter covers stacks, queues, and deques and how they are used; the only excluded material is the **internal implementation details of `std::deque<>`** (the array-of-pointers layout and the index arithmetic that goes with it). Everything else is fair game.

9.1 Introduction to Stacks

- A **stack** is a **one-ended** linear data structure with **last-in, first-out (LIFO)** access.
- Elements come out in the **opposite order** they went in.
- **Real-life analogy:** a stack of books on a table. You add to the top, and you must remove from the top. You cannot reach the bottom book without removing everything on top of it.

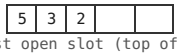
Function	Behavior
`.push(val)`	Adds `val` to the top of the stack
`.pop()`	Removes the top element from the stack
`.top()`	Returns a reference to the top element
`.size()`	Returns the number of elements in the stack
`.empty()`	Returns whether the stack is empty

Practical uses of stacks

- Text editor **undo** buttons (revert the most recent change first).
- Compilers checking for **matching parentheses/braces** (every `{` has a matching `}` in the right place).
- **Recursion**: keeping track of the data of previous function calls.
- **Graph searching**, such as depth-first search (chapter 19).

9.1.1 Implementing a stack using an array

- Use an array plus a pointer `top_ptr` to the **first empty position past the last element**, and a `base_ptr` to the start.



base_ptr at the left end, top_ptr at the first open slot (top of stack, where elements are added and removed)

- **push:** write at `top_ptr`, then increment it (allocating new space if needed).
- **pop:** just decrement `top_ptr`.
- **top:** dereference `top_ptr - 1`.
- **size:** `top_ptr - base_ptr` works because array elements are contiguous.
- **empty:** check `base_ptr == top_ptr`; if `top_ptr` is at the first position, that position must be empty.

We could check that an element actually exists before `.top()`, but the STL's `std::stack<>` **omits that check for efficiency**. `.top()` would be slower if it validated every call, so it is the programmer's job not to call `.top()` on an empty stack.

Method	Implementation	Complexity
`.push(val)`	Add at `top_ptr`, then increment `top_ptr`	`O(1)` with no reallocation, `O(n)` if reallocating
`.pop()`	Decrement `top_ptr`	`O(1)`
`.top()`	Dereference `top_ptr - 1`	`O(1)`
`.size()`	Compute `top_ptr - base_ptr`	`O(1)`
`.empty()`	Check `base_ptr == top_ptr`	`O(1)`

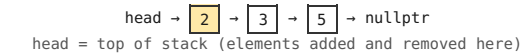
Footnote: using **amortized analysis**, `.push()` is an amortized `O(1)` operation. Amortized analysis is covered in chapter 12.

```
template <typename T>
class Stack {
  T* data;          // pointer to array
  T* top_ptr;       // pointer to first open position
  size_t capacity;  // array capacity
public:
  Stack(size_t num = 4);   // capacity defaults to 4
  ~Stack();
  void push(T val);
  void pop();
  T& top();
  size_t size();
  bool empty();
};

template <typename T>
Stack<T>::Stack(size_t num) : capacity{num} {
  data = (num ? new T[num] : nullptr);   // allocate array memory
  top_ptr = data;                        // set top_ptr to top of data
} // Stack()

template <typename T>
Stack<T>::~Stack() { delete[] data; }

template <typename T>
void Stack<T>::push(T val) {
  if (top_ptr - data >= capacity) {          // if current array is full
    T* temp = new T[capacity * 2];           // double capacity
    for (size_t i = 0; i < capacity; ++i) {  // copy elements over
      temp[i] = data[i];
    } // for i
    top_ptr = &temp[capacity];               // reset top_ptr to new array
    capacity *= 2;                           // update capacity
    std::swap(temp, data);                   // set data to new array
    delete[] temp;                           // deallocate old array
  } // if
  *top_ptr++ = val;                          // write new value at top_ptr
} // push()

template <typename T> void   Stack<T>::pop()   { --top_ptr; }
template <typename T> T&     Stack<T>::top()   { return *(top_ptr - 1); }
template <typename T> size_t Stack<T>::size()  { return top_ptr - data; }
template <typename T> bool   Stack<T>::empty() { return data == top_ptr; }
```

9.1.2 Implementing a stack using a linked list

- A **singly**-linked list is enough: no `prev` pointers are needed since we only operate on one end.

- Treat the **head as the top of the stack**, because adding and removing at the head is more efficient than at the tail.



head = top of stack (elements added and removed here)

- **push:** insert at the beginning of the list. The work does not depend on list size, so `θ(1)`.
- **pop:** set `head = head->next` and deallocate the old head, `θ(1)`.
- **top:** return a reference to the head node's data, `θ(1)`.
- **size:** counting nodes each time is `θ(n)`. Storing size in a **member variable** makes it `θ(1)`, at the cost of updating it on every insertion and deletion.
- **empty:** check `head == nullptr` (or `size == 0`), `θ(1)`.

Method	Implementation	Complexity
`.push(val)`	Insert `val` at the front of the list	`θ(1)`
`.pop()`	Delete the node at the front of the list	`θ(1)`
`.top()`	Return a reference to the head node's data	`θ(1)`
`.size()`	Track size internally, or count nodes each time	`θ(1) if stored internally, θ(n) if counted`
`.empty()`	Check `head == nullptr`	`θ(1)`

```
template <typename T>
class Stack {
  struct Node {
    T val;
    Node* next;
    Node(T val_in) : val{val_in}, next{nullptr} {}
  };
  Node* head;
  size_t sz;
public:
  Stack();  ~Stack();
  void push(T val);   void pop();
  T& top();  size_t size();  bool empty();
};

template <typename T>
Stack<T>::Stack() : head{nullptr}, sz{0} {}

template <typename T>
Stack<T>::~Stack() {
  Node* temp;
  while (head != nullptr) {   // destructor walks the list
    temp = head->next;        // and deallocates every node
    delete head;
    head = temp;
  } // while
} // ~Stack()

template <typename T>
void Stack<T>::push(T val) {
  Node* new_node = new Node{val};   // insert node at head
  new_node->next = head;
  head = new_node;
  ++sz;
} // push()

template <typename T>
void Stack<T>::pop() {              // remove node at head
  Node* temp = head;
  head = temp->next;
  --sz;
  delete temp;
} // pop()

template <typename T> T&     Stack<T>::top()   { return head->val; }
template <typename T> size_t Stack<T>::size()  { return sz; }
template <typename T> bool   Stack<T>::empty() { return head == nullptr; }
```

Array vs linked list for a stack. Both support every stack operation in `θ(1)`, but the **array version is usually slightly faster** because its constant factor is smaller. The linked list version also has **higher memory overhead**, since a pointer is stored with every element.

9.2 The STL Stack Container

- `#include <stack>` and declare a `std::stack<>`.

Function	Behavior
`.push(val)`	Adds `val` to the top of the stack
`.pop()`	Removes the top element (undefined behavior if empty)
`.top()`	Returns a reference to the top element (undefined behavior if empty)
`.size()`	Returns the number of elements
`.empty()`	Checks if the stack is empty

Retrieval and removal are separate. `.top()` returns a reference but does **not** remove. `.pop()` removes but does **not** return. To get and remove the top element you must call both.

```
std::stack<int32_t> s;            // initializes a stack named 's'
s.push(5);                        // pushes 5
s.push(3);                        // pushes 3
int32_t x = s.top();              // x stores 3
s.pop();                          // 3 is removed
s.top() = 4;                      // top element changed from 5 to 4
s.pop();                          // 4 is removed
std::cout << s.size() << '\n';    // stack is empty, prints 0
```

9.3 Introduction to Queues

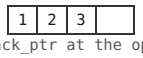
- A **queue** is a linear data structure with **first-in, first-out (FIFO)** access.
- Elements come out in the **same order** they went in.
- **Real-life analogy:** an office hours queue. You join the back of the line; the instructor helps whoever has waited longest.
- **enqueue** = add to the back (`.push()` in C++). **dequeue** = remove from the front (`.pop()` in C++).

Function	Behavior
`.push(val)`	Adds `val` to the back of the queue
`.pop()`	Removes the element at the front of the queue
`.front()`	Returns a reference to the element at the front
`.size()`	Returns the number of elements
`.empty()`	Checks if the queue is empty

Stacks use `.top()` to retrieve the next element; queues use `.front()`. Every other method name is identical.

9.3.1 Implementing a queue using an array (circular buffer)

- A queue is harder than a stack because operations happen on **both ends**.
- A **circular buffer** lets elements loop off the end of the array back to the beginning, which supports efficient insertion and deletion on opposite ends.
- Two pointers: `front_ptr` points to the element at the front, `back_ptr` points to the **first open position** at the back.



front_ptr at 1 (next to be popped), back_ptr at the open slot after 3 (most recently added)

- With a stack, `top_ptr` reaching the end meant "full". **That assumption fails for a queue**, because the first element is not guaranteed to be at the leftmost position (popping moves the front rightward).
- **push:** write at `back_ptr`, increment it, and **wrap around** to index 0 if it runs off the end.
- **pop:** move `front_ptr` forward one position. The old value is **not physically deleted**: the queue only looks between front and back, so it can never access that value again. Clearing it would be wasted work; it is overwritten when that index is needed again.

`[3][5][2][ ]` **`.pop()`** → `[x][5][2][ ]` **`.push(9)`** → `[x][5][2][9]` (back_ptr wraps to index 0) **`.push(4)`** → `[4][5][2][9]`

- **Full condition:** if incrementing `back_ptr` makes it equal `front_ptr`, the array is at capacity and must be reallocated.
- During reallocation, elements are copied **from front to back** into the larger array, so the front element ends up at index 0. In the example above the new array holds [5, 2, 9, 4].

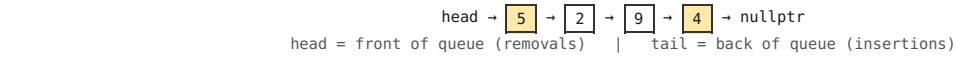
Method	Implementation	Complexity
`.push(val)`	Write at `back_ptr` and increment it, wrapping to the start if needed. If `back_ptr` is ever incremented onto `front_ptr`, allocate a larger array and copy elements over in order from front to back.	`Θ(1)` with no reallocation, `Θ(n)` if reallocating
`.pop()`	Increment `front_ptr`	`Θ(1)`
`.front()`	Dereference `front_ptr` and return its value	`Θ(1)`
`.size()`	If `back_ptr` is at a higher address than `front_ptr`, return the difference; otherwise return `capacity + back_ptr - front_ptr`	`Θ(1)`
`.empty()`	Check `back_ptr == front_ptr`	`Θ(1)`

Footnote: using amortized analysis, `.push()` is an amortized `Θ(1)` operation (chapter 12).

```
template <typename T>
class Queue {
  T* data;  T* front_ptr;  T* back_ptr;  size_t capacity;
public:
  Queue(size_t num = 4);   // capacity defaults to 4
  ~Queue();
  void push(T val);  void pop();
  T& front();  size_t size();  bool empty();
};

template <typename T>
Queue<T>::Queue(size_t num) : capacity{num} {
  data = (num ? new T[num] : nullptr);
  front_ptr = back_ptr = data;
} // Queue()

template <typename T>
Queue<T>::~Queue() { delete[] data; }

template <typename T>
void Queue<T>::push(T val) {
  *back_ptr++ = val;                       // write val at back_ptr
  if (back_ptr >= data + capacity) {       // loop around if off end
    back_ptr = data;
  } // if
  if (back_ptr == front_ptr) {             // back hit front: reallocate
    T* temp = new T[capacity * 2];         // double capacity
    for (size_t i = 0; i < capacity; ++i) {
      temp[i] = *front_ptr++;              // copy starting from front
      if (front_ptr == data + capacity) {  // loop around if off end
        front_ptr = data;
      } // if
    } // for i
    front_ptr = temp;                      // reset front_ptr to new array
    back_ptr = &temp[capacity];            // reset back_ptr to new array
    capacity *= 2;
    std::swap(temp, data);                 // set data to new array
    delete[] temp;                         // deallocate old array
  } // if
} // push()

template <typename T>
void Queue<T>::pop() {
  ++front_ptr;
  if (front_ptr >= data + capacity) {      // loop around if off end
    front_ptr = data;
  } // if
} // pop()

template <typename T> T& Queue<T>::front() { return *front_ptr; }

template <typename T>
size_t Queue<T>::size() {
  if (back_ptr >= front_ptr) { return back_ptr - front_ptr; }
  else { return capacity + back_ptr - front_ptr; }
} // size()

template <typename T> bool Queue<T>::empty() { return back_ptr == front_ptr; }
```

9.3.2 Implementing a queue using a linked list

- Same idea as the list-based stack, except elements are added at the **tail** and removed at the **head**.



Method	Implementation	Complexity
`.push(val)`	Insert `val` at the back of the list	`O(1) with a tail pointer, O(n) without`
`.pop()`	Delete the head node of the list	`O(1)`
`.front()`	Return a reference to the head node's data	`O(1)`
`.size()`	Track size internally, or count nodes each time	`O(1) if stored internally, O(n) if counted`
`.empty()`	Check `head == nullptr`	`O(1)`

```
template <typename T>
class Queue {
  struct Node {
    T val;
    Node* next;
    Node(T val_in) : val{val_in}, next{nullptr} {}
  };
  Node* head;  Node* tail;  size_t sz;
public:
  Queue();  ~Queue();
  void push(T val);  void pop();
  T& front();  size_t size();  bool empty();
};

template <typename T>
Queue<T>::Queue() : head{nullptr}, tail{nullptr}, sz{0} {}

template <typename T>
Queue<T>::~Queue() {
  Node* temp;
  while (head != nullptr) { temp = head->next; delete head; head = temp; }
} // ~Queue()

template <typename T>
void Queue<T>::push(T val) {
  Node* new_node = new Node{val};   // insert node at tail
  if (tail != nullptr) { tail->next = new_node; }
  else { head = new_node; }         // empty list: new node is also head
  tail = new_node;
  ++sz;
} // push()

template <typename T>
void Queue<T>::pop() {
  Node* temp = head;                // remove node at head
  head = temp->next;
  --sz;
  delete temp;
  if (head == nullptr) { tail = nullptr; }   // queue is now empty
} // pop()

template <typename T> T&     Queue<T>::front() { return head->val; }
template <typename T> size_t Queue<T>::size()  { return sz; }
template <typename T> bool   Queue<T>::empty() { return head == nullptr; }
```

9.4 The STL Queue Container

- `#include <queue>` and declare a `std::queue<>`.

Function	Behavior
`.push(val)`	Adds `val` to the back of the queue
`.pop()`	Removes the element at the front (undefined behavior if empty)
`.front()`	Returns a reference to the front element (undefined behavior if empty)
`.size()`	Returns the number of elements
`.empty()`	Checks if the queue is empty

Like `std::stack<>`, retrieval and removal are separate: `.front()` returns without removing, `.pop()` removes without returning.

```
std::queue<int32_t> q;            // initializes a queue named 'q'
q.push(5);                        // pushes 5
q.push(3);                        // pushes 3
int x = q.front();                // x stores 5
q.pop();                          // 5 is removed
q.front() = 4;                    // front element changed from 3 to 4
q.pop();                          // 4 is removed
std::cout << q.size() << '\n';    // queue is empty, prints 0
```

9.5 Solving Problems Using Stacks and Queues

EXAMPLE 9.1 **9.5.1 Implementing a queue using two stacks**

Given two stacks supporting `.top()`, `.push()`, `.pop()`, `.size()` and no other containers, build a queue supporting `.front()`, `.push()`, `.pop()`, `.size()`.

- A single stack cannot simulate a queue: it is **one-ended**, so there is no way to reach the bottom. A queue must manage **both** ends.
- With **two** stacks there are two places to add or remove: the top of stack 1 and the top of stack 2. Treat one stack as the **front** of the queue (removals) and the other as the **back** (insertions).

```
.push(x): stackBack.push(x);
.front():  return stackFront.top();
.pop():    stackFront.pop();
.size():   return stackFront.size() + stackBack.size();
```

Problem with the naive version: push 1 and immediately pop. The 1 sits in the back stack while the front stack is empty, so there is nothing to pop. `.pop()` and `.front()` break whenever all elements are in the back stack.
Fix: if `.pop()` or `.front()` is called while the front stack is empty, **move every element from the back stack to the front stack**. All of them must move, because the element we want is at the bottom of the back stack and everything above it must come off first. Transferring through a stack also reverses the order, which is exactly what turns LIFO into FIFO.

```
QueueWithTwoStacks::front() {          QueueWithTwoStacks::pop() {
  if (stackFront.empty()) {              if (stackFront.empty()) {
    while (!stackBack.empty()) {           while (!stackBack.empty()) {
      stackFront.push(stackBack.top());      stackFront.push(stackBack.top());
      stackBack.pop();                       stackBack.pop();
    } // while                              } // while
  } // if                                   } // if
  return stackFront.top();                 stackFront.pop();
} // front()                             } // pop()
```

Trace for `push(1) push(2) push(3) pop() push(4) push(5) pop() pop() pop()`:

- Push 1, 2, 3: back stack holds 1, 2, 3 (3 on top). Front stack empty.
- First pop: front is empty, so transfer everything. Front stack now holds 3, 2, 1 with **1 on top**. Pop removes 1.
- Push 4, 5: they go into the back stack. Front stack still holds 2, 3.
- Next two pops: front stack is not empty, so 2 then 3 come off with no transfer.
- Third pop: front stack is now empty, so transfer 4 and 5 over, then pop 4.

Complexity: `.push()` and `.size()` are $\Theta(1)$. `.front()` and `.pop()` are worst-case $\Theta(n)$, since they may move every element between stacks. That worst case only happens when the front stack is empty, and it happens so rarely that **amortized analysis** shows both can essentially be treated as $\Theta(1)$ (chapter 12).

EXAMPLE 9.2 **9.5.2 Sorting a stack**

Sort a stack using only `.top()`, `.push()`, `.pop()`, `.size()`, plus one auxiliary stack and no other containers.

Idea: keep the auxiliary stack sorted at all times by making sure **bigger values are pushed toward the bottom of aux**.

1. Save the top of the input stack in a variable (the **current element**).
2. Move every element in aux that is **smaller** than the current element back into the input stack.
3. Once aux only holds elements larger than the current element, push the current element into aux.

```
while (!input.empty()) {
  curr = input.top();
  input.pop();
  while (!aux.empty() && aux.top() < curr) {
    input.push(aux.top());
    aux.pop();
  } // while
  aux.push(curr);
} // while
```

Trace with input 2, 5, 3, 1, 4 (2 on top):

- curr = 2: aux empty, push 2. aux = [2].
- curr = 5: move 2 back to input, push 5, then 2 returns on top. aux = [5, 2] with 2 on top.
- curr = 3: move 2 back, push 3, 2 returns. aux = [5, 3, 2].
- curr = 1: 1 is smaller than everything in aux, push directly. aux = [5, 3, 2, 1].
- curr = 4: move 1, 2, 3 back, push 4, then return 1, 2, 3. aux = [5, 4, 3, 2, 1].

Time $\Theta(n^2)$ in the worst case, which occurs when the input stack is **already sorted**: every push into aux requires emptying it. Pushing the 2nd value moves 1 element out and back, the 3rd moves 2, and the nth moves n - 1, giving $1 + 2 + \ldots + (n - 1) = \Theta(n^2)$. **Auxiliary space $\Theta(n)$** for the extra stack.

EXAMPLE 9.3 **9.5.3 Sorting a queue**

Sort a queue using only `.front()`, `.push()`, `.pop()`, `.size()` and **no additional containers**.

- A queue is **not** single-ended, so no auxiliary container is needed. Popping from the front and repushing onto the back eventually exposes every element, using only $\Theta(1)$ auxiliary space.

Algorithm

1. Make repeated passes: pop from the front and reinsert at the back. During each pass track the **smallest value seen so far** (`curr_min`). When you find a new smallest, hold it out and push the old minimum to the back instead.
2. At the end of each pass, push the minimum found to the back. That value is now in sorted position and is **ignored on all future passes**.
3. Repeat until the queue is sorted.

Trace starting from front-to-back 2, 5, 3, 1, 4:

- Pass 1: hold 2 as curr_min. 5 is not smaller, cycle it to the back. 3 is not smaller, cycle it. 1 **is** smaller, so push 2 to the back and hold 1. 4 is not smaller, cycle it. Pass ends: push 1 to the back and fix it. Order: 4, 5, 3, 2, | 1.
- Pass 2: hold 4. 5 is not smaller, cycle. 3 is smaller, push 4 back and hold 3. 2 is smaller, push 3 back and hold 2. 1 is already fixed, so ignore and cycle. Pass ends: 2 is fixed. Order: 5, 4, 3, | 1, 2.
- Pass 3 fixes 3, pass 4 fixes 4. At that point 5 is the only unfixed element, so it must be the largest; pop it and push it to the end.
- Final: 1, 2, 3, 4, 5.

Time $\Theta(n^2)$: about n passes, each making n comparisons. **Auxiliary space $\Theta(1)$**: the sorting is done in place using only the input queue.

EXAMPLE 9.4 **9.5.4 Evaluating reverse Polish notation**

In **reverse Polish notation** the operator follows its operands: "1 2 +" means 1 + 2, and "(3 - 4) * 5" is written "3 4 - 5 *". Given a valid expression as a vector of strings with operators "+", "-", "*", "/", evaluate it. Assume it always evaluates and never divides by zero.

Why a stack: each operator acts on the results of the **most recent** expressions before it, which is exactly LIFO behavior.

1. Iterate over the input tokens.
2. A number gets pushed onto the stack.
3. An operator pops the **top two** values, applies the operator, and pushes the result back.
4. At the end, the single remaining value is the answer.

Trace for ["2", "3", "*", "4", "5", "*", "+"]: push 2, push 3, then "*" pops 2 and 3 and pushes **6**; push 4, push 5, then "*" pops 4 and 5 and pushes **20**; then "+" pops 6 and 20 and pushes **26**. Answer: **26**.

```
int32_t apply_operation(int32_t a, int32_t b, char op) {
  switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return a / b;
    default: throw std::invalid_argument{"Invalid operation"};
  } // switch
} // apply_operation()

int32_t evaluate_reverse_polish_notation(std::vector<std::string>& tokens) {
  std::stack<int32_t> values;
  for (const std::string& token : tokens) {
    if ((token == "+" || token == "-" || token == "*" || token == "/")) {
      int32_t value1 = values.top();  values.pop();
      int32_t value2 = values.top();  values.pop();
      values.push(apply_operation(value2, value1, token[0]));
    } // if
    else {
      values.push(std::stoi(token));  // stoi() converts string to integer
    } // else
  } // for token
  return values.top();
} // evaluate_reverse_polish_notation()
```

Note the operand order: the first value popped is the *right* operand, so the call is `apply_operation(value2, value1, ...)`. This matters for "-" and "/".

Time `O(n)` (one pass over the input), **auxiliary space `O(n)`** for the stack.

9.5.5 **Balancing parentheses**

Given a string of `()`, `[]`, and `{}`, determine whether the parentheses match.

Balanced	Not balanced
`"{}[]"`, `"[({})]"`, `"(){[]{}}"`	`"}{{}}{"`, `"[(])"`, `"(()"`

Rule: each closing parenthesis must match the **most recent unmatched opening** parenthesis before it.

The counter approach, and why it fails

1. Keep three counters: `left_paren`, `left_brack`, `left_brace`. Scan left to right.
2. On an opening character, increment that type's counter.
3. On a closing character, decrement that type's counter. If it is already zero there is no unmatched opener, so the string is unbalanced and we exit.
4. If all counters are exactly zero at the end, call it balanced.

This is wrong. For `"[(])"` every type appears an equal number of times, so the counters say balanced, but the **order** is invalid. The count *and* the position both have to match.

The stack approach

1. Initialize a stack and scan the string left to right.
2. On an **opening** character, push it onto the stack.
3. On a **closing** character, look at the top of the stack. If the stack is **empty**, there is no matching opener, so the string is unbalanced. If the top is an opener of the **same type**, pop it and continue. Otherwise the order is wrong and the string is unbalanced.
4. At the end, check the stack is **empty**. A non-empty stack means some openers were never matched, so the string is unbalanced.

Trace on `"[(])"`: push `[`, push `(`, then see `]`. The top is `(`, which is a different type, so the string is immediately known to be unbalanced. Order no longer breaks the algorithm.
Trace on `"[()]"`: push `[`, push `(`, see `)` which matches the top so pop it, see `]` which matches `[` so pop it. String ends with an empty stack, so it is balanced.
Time and auxiliary space both worst-case `O(n)`, where *n* is the string length.

9.6 Search Applications of Stacks and Queues

- Searching problems appear everywhere: shortest path in a map, escaping a maze with obstacles, word ladder puzzles where you change one letter at a time.
- A standard searching problem gives you a **starting state**, a set of **next states** reachable from the current state, and a **destination state**.
- The states you have discovered but not yet fully explored are your **search frontier**. A stack or a queue holds that frontier.
- Whether you use a stack or a queue changes the **order** in which states are encountered.

Stack frontier = depth-first search (DFS)

- Rushes down each viable path as far as possible before backtracking and trying alternatives.
- All neighboring states discovered are pushed onto a stack, and the state on **top** (most recently discovered) is explored next.
- Finds very deep solutions quickly, typically with **lower memory** than a queue-based search when paths are long but narrow.

Queue frontier = breadth-first search (BFS)

- Checks all neighboring states **level by level**: first everything one state away, then two away, then three away.
- All neighbors discovered are pushed into a queue, and the state at the **front** (oldest unprocessed) is explored next.
- Finds the path passing through the **fewest intermediary states**, that is, a **shortest path**, provided travel between any two states costs the same and is unweighted.

Search container	Stack	Queue
Exploration order	Depth-first (goes deep quickly)	Breadth-first (goes wide, explores outward)
Overall memory usage	Typically lower	Typically higher
Guarantees shortest path?	No	Yes, if unweighted
Typical problem family	Find any path to a solution	Find the path with the minimal number of moves

Most search and pathfinding problems are built on one of these two strategies with small modifications. Full coverage of these algorithms comes with graph algorithms in chapter 19.

9.7 The STL Deque Container

- A **deque** (pronounced "deck"), or **double-ended queue**, supports the functionality of both a stack and a queue.
- Unlike a vector, which is only efficient at the back, a deque is efficient at **both ends**: `.push_front()`, `.pop_front()`, `.push_back()`, `.pop_back()`.

What the C++ standard requires of a deque

1. `O(1)` insertion and removal at **both** the front and the back.
2. `O(1)` **random access** using `operator[]`.
3. Insertion or deletion at **either end must never invalidate** pointers or references to the rest of the elements.

Why the obvious implementations fail

- **Linked list:** supports Θ(1) insert and remove at both ends with a tail pointer, but **cannot support `operator[]` in Θ(1)**, since lists have no random access. Fails requirement 2.
- **Circular array:** gives Θ(1) random access and avoids shifting on front operations, but it occasionally needs **reallocation**, which moves every element to a new location and **invalidates existing pointers**. Fails requirement 3.

NOT TESTED: DEQUE IMPLEMENTATION DETAILS

Per the course directions you do not need to know how `std::deque<>` is implemented internally. Included for completeness. (The exact implementation may differ across libraries as long as it obeys the standard; the one below is from a GCC standard library implementation.)

- A deque is an **array of pointers to other arrays**. Each **inner array** has a constant size, usually a power of two, and holds the actual data. The **outer array** holds the pointers.
- The **last** inner array fills toward the back; the **first** inner array fills toward the front.
- When the first or last inner array fills up, a new inner array is allocated at the next available position of the outer array.
- If the entire outer array fills up, the outer array is reallocated to a larger capacity, like a vector. Unlike a vector, the extra space is **split between both ends**, which keeps insertion constant time on both sides.
- This solves both earlier problems: inner arrays are never reallocated, so elements keep their memory addresses (no pointer invalidation), and the layout still allows `O(1)` random access.
- **Index arithmetic for `operator[]`:** add the requested index to the number of unused slots in the first inner array. Then
  • **divide** by the inner array capacity (integer division) to get which inner array holds the value;
  • take the **modulo** with the inner array capacity to get the position inside that inner array.
  Example: index 7 with 2 unused slots at the front and inner arrays of capacity 4: 7 + 2 = 9, then 9 / 4 = 2 and 9 % 4 = 1, so the element is at index 1 of inner array 2.

`std::deque<>` interface (`#include <deque>`)

Constructors

```
std::deque<int32_t> d1;                    // default, empty
std::deque<int32_t> d2{1, 2, 3};           // initializer list, {1,2,3}
std::deque<int32_t> d3 = {1, 2, 3};
std::deque<int32_t> d4{d3};                // copy constructor, {1,2,3}
std::deque<int32_t> d5(5);                 // size 5, each value-initialized to 0
std::deque<int32_t> d6(5, 1);              // size 5, each initialized to 1
std::deque<int32_t> d7(d3.begin(), d3.begin() + 2);  // range [begin,end), {1,2}
```

Core operations

- `.size()`, `.empty()`, `.clear()` (leaves size 0)
- `.front()`, `.back()`: reference to first / last element (undefined behavior if empty)
- `.push_back(const T& val)`, `.push_front(const T& val)`: size increases by 1
- `.emplace_back(Args&&... args)`, `.emplace_front(Args&&... args)`: construct in place at the back / front; return a reference since C++17
- `.pop_back()`, `.pop_front()`: size decreases by 1 (undefined behavior if empty)
- `.insert(pos, val)` inserts directly before `pos`, returns iterator to the new element
- `.insert(pos, n, val)` n copies, returns iterator to the first added
- `.insert(pos, first, last)` copies the range [first, last), returns iterator to the first added
- `.insert(pos, std::initializer_list<T> init)`, returns iterator to the first added
- `.emplace(const_iterator pos, Args&&... args)` constructs in place before `pos`, returns iterator to it
- `.erase(pos)` returns iterator to the element after the erased one; `.erase(first, last)` erases the range and returns the iterator after the last erased
- `operator[](size_t n)` returns a reference to the element at index n

Iterators (same idea as a vector)

Function	Behavior
`.begin()` / `.end()`	Random access iterator to the first element / one past the last element
`.cbegin()` / `.cend()`	Constant random access versions of the above
`.rbegin()` / `.rend()`	Reverse iterator to the last element / one before the first element
`.crbegin()` / `.crend()`	Constant reverse versions of the above

Deque vs vector

- Like a vector, inserting or deleting in the **middle** requires shifting elements (to open a space or close a gap), so it is not Θ(1).
- **Reallocation is cheaper for a deque:** a vector copies all its data to a new larger array, while a deque only copies its **outer array of pointers**, since the inner arrays hold the data and are never reallocated.
- **`operator[]` is faster on a vector.** Both are Θ(1), but the deque's constant is larger: a vector needs one addition (`arr[i]` is `*(arr + i)`) while a deque needs more math to find the address.
- Deque elements are **not always contiguous** in memory, so iterating a deque is typically slower than iterating a vector, which exploits **caching** better.

9.8 Container Adaptors

- **Sequence containers:** vector, list, deque. Their data can be sequentially accessed in an established order.
- **Container adaptors:** stack, queue, and priority queue (chapter 10). They are **not full container classes**, but interfaces that adapt a standard container by **modifying or restricting** its functionality.
- In the STL, **container adaptors do not support iteration**.

The STL stack and queue are built on a `std::deque<>` by default, not on an array or a linked list. A `std::stack<>` is a deque restricted to one end; a `std::queue<>` is a deque that forces insertion and removal on different ends.

- Functionally you could just use a deque: use only `.push_back()` and `.pop_back()` for stack behavior, or `.push_back()` and `.pop_front()` for queue behavior.
- But it is **better practice to declare the container that matches your need**. It improves readability and reduces mistakes such as inserting or removing from the wrong end. Choose a deque only when you actually need double-ended behavior that a stack or queue does not provide.
- Because adaptors only restrict the interface of the underlying container, there is **no performance difference** between using a stack or queue and using the deque directly.

	STL Stack (`std::stack<>`)	**STL Queue (`std::queue<>`)**
Default underlying container	`std::deque<>` permitting only `.push_back()` and `.pop_back()`	`std::deque<>` permitting only `.push_back()` and `.pop_front()`
Optional underlying container (must be explicitly specified)	`std::list<>` with only `.push_back()` / `.pop_back()`, **or** `std::vector<>` with only `.push_back()` / `.pop_back()`	`std::list<>` with only `.push_back()` / `.pop_front()`

- Requirement: a **stack's** underlying container must support `.push_back()` and `.pop_back()`; a **queue's** must support `.push_back()` and `.pop_front()`.
- **A `std::queue<>` cannot use a `std::vector<>`**, because vectors have no `.pop_front()` and cannot give efficient insertion and removal at different ends.
- To specify the underlying container, put the container type right after the data type:

```
std::stack<int32_t, std::vector<int32_t>> s;   // stack backed by a vector
std::queue<int32_t, std::list<int32_t>>   q;   // queue backed by a list
```

If you do not specify one, `std::deque<>` is used. For most scenarios a deque is the ideal underlying container for stacks and queues, which is why the STL uses it by default.

Facts worth memorizing

- Stack = LIFO, uses `.top()`. Queue = FIFO, uses `.front()`. Everything else is named the same.
- Both stacks and queues support all their operations in $\theta(1)$ on either an array or a linked list.
- Retrieval and removal are always separate calls in the STL.
- Getting to the **most recently added** element of a `std::queue<>` and removing it takes $\theta(n)$: you must cycle the other n - 1 elements through. Doing so needs only $\theta(1)$ auxiliary space, since you can pop from the front and repush to the back.
- Circular buffer is **full** when incrementing `back_ptr` lands it on `front_ptr`, not when it runs off the end of the array.
- Reversing a queue: push everything into a stack, then pop it all back into the queue.
- A queue can simulate a stack: after each push, cycle every previously existing element from front to back, so the newest element is always at the front. Each push is then $\theta(n)$, making n pushes $\theta(n^2)$.
- Emulating recursion without recursion needs LIFO access, so use a stack or a vector, not a queue.
- Deques give $\theta(1)$ at the ends only; insertion in the middle can be $\theta(n)$.
- A queue backed by a singly-linked list can still report `.size()` in $\theta(1)$ by tracking size internally.